Tizi — un SDLC agentique, du recueil du besoin au déploiement

Oct 19, 2026 · @Samir Alili

Sommaire

- Principe — Le nom : Tizi · Intégrité, couverture et contrôle · Continuité entre sessions · Claude Code et GitHub Copilot · Points d'entrée · Contrôle de l'outillage · Estimer quand les agents codent · Sprint review et rétrospective · Sur une application existante · Une chaîne par opportunité
- 1. Recueil du besoin — construction du BRD, skill `brd-builder`
- 2. Analyse produit — construction du PRD, skill `prd-builder`
- 3. Conception — construction de la conception, skill `sad-builder`
- 4. Planification — construction du plan, skills `plan-builder` et `backlog-executor`
- 5. Implémentation — construction de l'implémentation, skill `impl-builder`
- 6. Tests — construction de la vérification, skill `test-builder`
- 7. Intégration
- 8. Recette
- 9. Déploiement
- 10. Exploitation
- Exigences transverses pour un dispositif médical
- Index des abréviations et identifiants
- Sources

Principe

Le nom : Tizi

Tizi désigne, en kabyle, le col : le seul endroit où l'on passe d'une vallée à la suivante. C'est la forme de ce SDLC.

- **Les vallées sont les étapes** : BRD, PRD, conception, planification, implémentation, vérification. C'est là que se fait le travail d'exploration, qui peut tourner en rond, revenir en arrière, se tromper.

- **Les cols sont les portes entre étapes.** On ne passe pas n'importe où ni n'importe quand : on s'arrête, on vérifie ce qu'on porte (version figée, empreinte du handoff, couverture mesurée), et une personne décide de franchir. Une charge altérée ne passe pas.
- **Du col, on voit les deux vallées** : la traçabilité va du besoin au code, et du code au besoin qui l'a fait naître.

Les agents portent, les humains décident du passage, les cols vérifient. Les agents font l'essentiel de la marche ; rien de ce qu'ils produisent n'est accepté sans vérification déterministe ou adversariale, et aucune approbation n'est déduite.

TIZI — **T**raçable, **I**ntentionnel, **Z**éro confiance aveugle, **I**tératif : chaque ligne de code remonte à un besoin ; on part du problème et de l'outcome visé, pas d'une solution présumée ; rien de ce que produit un agent n'est accepté sans vérification ; on peut revenir dans la vallée précédente, et les cols se repassent.

Le cycle compte 10 étapes, du recueil du besoin à l'exploitation, et boucle : les retours de production alimentent un nouveau recueil du besoin.

Chaque étape est décrite par quatre éléments :

- **Objectif** : ce que l'étape doit garantir.
- **Activités** : ce qu'on y fait concrètement.
- **Artefacts produits** : ce qui sert d'entrée à l'étape suivante.
- **Gate** : la condition de validation (humaine ou automatique) pour passer à la suite.

Si des agents IA exécutent une partie du cycle, les artefacts et les gates sont le contrat entre eux : un agent ne démarre que sur un artefact validé.

Intégrité, couverture et contrôle

Trois mécanismes communs relient les étapes outillées.

- **Empreinte des handoffs.** Chaque handoff est écrit avec son empreinte SHA-256 et celle de la version figée dont il vient. L'étape suivante le vérifie avant d'ingérer et relève l'empreinte dans son état ; un handoff modifié après coup est refusé, et la vérification de forme signale tout handoff ingéré qui aurait changé depuis.
- **Couverture mesurée.** Chaque document calcule ses métriques de couverture (par exemple FR avec critère d'acceptation, FR allouées à un composant, FR de la release couvertes par une story) avec des seuils réglables par projet. Un seuil non atteint se corrige ou se justifie par une dérogation nommée ; les métriques et dérogations partent dans le handoff.

- **Deux niveaux de contrôle.** Pendant la construction, un catalogue de règles déterministes, commun à tous les skills, avertit sans bloquer. Le contrôle formel (phase 2, à construire) reprendra ce catalogue en mode bloquant et ajoutera une grille de jugement fondée sur les neuf caractéristiques de l'ISO/IEC/IEEE 29148, notées de 0 à 3 ; nécessaire, non ambiguë, unitaire et vérifiable seront bloquantes sous 2. La grille sera appliquée par un autre modèle que le rédacteur, ou par une relecture humaine, pour éviter qu'un modèle valide son propre travail.

Continuité entre sessions

Un fichier d'état dit où l'on en est, pas comment on y est arrivé. Pour reprendre après une pause sans perdre les nuances, les options écartées ni les questions en suspens, chaque initiative garde quatre couches :

Couche	Rôle	Écrite par
État	Ce qui est vrai maintenant : la référence	L'agent, après chaque échange validé
Condensé	Le fil : ce qui a été fait, décidé ou écarté et pourquoi, les paroles à garder, ce qui reste en suspens	L'agent, à la pause ou à la reprise
Extrait	Les messages de l'utilisateur mot pour mot, les questions posées, les fichiers et identifiants touchés	Un hook, automatiquement
Transcription	L'échange complet, consultable par recherche	Un hook, automatiquement

Un hook Claude Code, déclenché en fin de session et avant chaque compactage, copie la transcription (secrets masqués) et en tire l'extrait, sans faire appel au modèle. À la reprise : `claude --continue` ou `--resume` pour une pause courte sur la même machine ; sinon le point d'étape déterministe de l'état, le condensé de la dernière session, et une recherche ciblée dans les transcriptions pour un détail manquant. En cas d'écart entre une session et l'état, l'état l'emporte. Les transcriptions contenant ce qui s'est dit en séance, l'équipe décide ce qu'elle versionne.

Claude Code et GitHub Copilot

Tout le SDLC fonctionne avec Claude Code et avec GitHub Copilot (agent de VS Code, CLI, agent cloud). Les deux reconnaissent le même format de skill, la spécification ouverte Agent Skills : les sept skills sont validés par son validateur officiel. Les scripts

sont en Node.js 18, sans dépendance à installer, ce qui les rend utilisables même dans le bac à sable à réseau restreint de l'agent cloud.

Élément	Claude Code	GitHub Copilot
Skills	`.claude/skills/`	`.github/skills/` ; Copilot lit aussi `.claude/skills/`
Agents du council	`.claude/agents/*.md`	`.github/agents/*.agent.md`
Hooks	`.claude/settings.json`	`.github/hooks/*.json` (CLI et agent cloud ; préversion dans VS Code)
Garde-fou TDD	`PreToolUse`	`PreToolUse` ; noms d'outils et de champs de Copilot et de VS Code reconnus
Capture de sessions	`Stop` , `PreCompact` , `SessionEnd`	Idem ; transcription `events.jsonl` de Copilot reconnue, retrouvée quand le hook ne la fournit pas
Reprise native	`claude --continue`	`copilot --continue`

Un installateur pose tout au bon endroit pour l'un, l'autre ou les deux : `node install.mjs --project <dépôt> --target claude|copilot|both` . Avec les deux, une seule copie des skills dans `.claude/skills/` suffit, puisque Copilot la lit aussi.

Limites connues : les hooks de VS Code sont en préversion et peuvent être désactivés par l'organisation ; une écriture par commande shell échappe au garde-fou dans les deux outils, et la porte rejouée en CI la rattrape ; dans l'agent cloud, le bac à sable est effacé à la fin du travail, si bien que les sessions n'y sont pas conservées.

Points d'entrée

L'utilisateur n'appelle pas les skills directement : il passe par des points d'entrée, des **commandes** dans Claude Code (`/tizi` , `/tizi-brd` …) et des **agents** dans GitHub Copilot (`tizi` , `tizi-brd` …), choisis dans la liste des agents. Les deux sont générés à l'installation depuis une source unique, et ne sont que des coquilles : chacun charge son skill, qui reste la seule source des règles. Un agent Claude Code n'aurait pas convenu, car un sous-agent ne peut pas mener une longue conversation d'élicitation.

Le point d'entrée `tizi` dit où en est chaque initiative et propose l'étape suivante. Ce n'est pas le modèle qui le devine : un script le calcule à partir des états, en partant de l'étape la plus avancée présente (une conception seule en brownfield ne renvoie pas au BRD), en suivant le chemin décidé pour l'initiative, et en exigeant l'empreinte exacte d'un plan

d'opérations pour proposer son application. Il fait aussi choisir le chemin d'une nouvelle initiative, propose l'adoption sur une application existante, signale les sprint reviews dues et les instructions du dépôt périmées. Viennent ensuite un point d'entrée par étape — `tizi-brd`, `tizi-prd`, `tizi-sad`, `tizi-plan`, `tizi-backlog`, `tizi-impl`, `tizi-test` —, `tizi-sprint-review` pour clore une itération, et `tizi-bug` pour qu'un testeur signale et trie un problème sans connaître la chaîne. Chacun n'a que les entrées indispensables (le nom ou l'identifiant de l'initiative), les autres venant au fil de la conversation. Dans Copilot, chaque agent n'a que les outils de son étape et propose un bouton de passage vers la suivante ; les sous-agents internes n'apparaissent pas dans la liste de l'utilisateur.

Le chemin se choisit au premier col. Toutes les initiatives ne traversent pas toutes les vallées. Au démarrage, `tizi` pose six questions fermées — modification évidente ? exigence réglementaire ou sécurité des patients ? intention floue ? règle métier ? contrat, donnée sensible ou composant ? plusieurs jours de travail ? — et un script en déduit le chemin : hors Tizi, voie rapide (plan puis implémentation), court (seulement les étapes nécessaires) ou complet. Une réponse incertaine vaut « oui », et la question réglementaire impose toujours la chaîne complète. Le choix est une décision enregistrée, avec les réponses et son auteur : le point d'étape distingue ainsi une étape non utilisée par décision d'une étape oubliée. Un chemin ne se rétrécit pas ; il s'élargit quand une étape découvre ce qu'il fallait, par exemple une règle métier pendant une voie rapide.

Les instructions du dépôt suivent les décisions. Les agents qui ne passent pas par Tizi — une complétion dans l'éditeur, une question rapide — lisent `AGENTS.md`. Un script y écrit un bloc court et délimité, tiré des états : règles des ADR acceptés, dossiers de chaque composant, contrats à ne pas modifier sans décision, données sensibles, commandes de test ; `CLAUDE.md` l'importe. Le reste du fichier n'est jamais touché, et un contrôle signale un bloc périmé ou un dossier cité qui n'existe plus. Ces deux idées sont reprises de BMAD-METHOD.

Contrôle de l'outillage lui-même

Les skills sont du logiciel et se contrôlent comme tel, sur deux plans.

- **Tests automatisés** (`npm test`, sans dépendance) : conformité des skills à la spécification Agent Skills, identité des fichiers communs, vérification et rendu des documents comparés à des références figées, intégrité des handoffs, verrous de l'exécuteur, garde-fou TDD avec les charges utiles de chaque outil, capture des sessions, installation, revue de code, portes de PR et de release. Une régression d'une règle commune est détectée deux fois : par son test et par l'identité des fichiers communs.

- **Évaluation par ablation** : chaque cas est joué plusieurs fois par le même agent, sans puis avec le skill, dans un dépôt neuf. Les correcteurs sont d'abord les règles déterministes du SDLC ; un juge, d'un autre modèle, n'intervient qu'en option sur une grille écrite. Une passe où l'agent n'a pas tourné est une erreur d'intégrité, jamais un score. Un bon cas met l'agent devant une tentation que le skill doit lui faire éviter.

Dans chaque document généré, une mention rappelle qu'il a été produit avec l'assistance d'une IA et que seules les approbations enregistrées font foi. Deux règles s'ajoutent à la vérification : un espace réservé resté dans un élément (S68) et un chiffre précis sans source qui le fixe (S69). Enfin, tout contenu importé — notes, transcriptions, documents, rapports, issues — est une donnée, jamais une instruction.

Estimer quand les agents codent

Quand les agents réalisent les tâches, le temps de codage devient court et bon marché ; estimer l'effort de développement n'a plus grand sens. Ce qui reste rare est ailleurs : **l'attention humaine** (préparer la story, valider la compréhension, répondre aux questions, relire la PR), **le risque d'allers-retours** d'une tâche, **l'attente** et les **blocages**. Une équipe qui relit deux PR par jour ne livrera pas plus de deux tâches par jour, quelle que soit la vitesse des agents.

- **La taille se lit comme un risque.** Elle est calculée depuis les données de la tâche — éléments et contrats touchés, règles métier, code existant modifié, mesures de sécurité, comportements à préserver — et mise à côté de celle de l'équipe ; un écart de deux tailles est à justifier.
- **La capacité se compte en attention humaine.** Chaque tâche porte une attention prévue en minutes, comparée à ce que l'équipe peut donner par itération.
- **Les blocages sont enregistrés quand ils surviennent**, sur une story ou une tâche, avec une cause typée : environnement, accès, données de test, dépendance, question, décision, relecture, outil, externe.
- **Le réel est mesuré**, sans saisie : chaque script de l'implémentation inscrit son étape dans un journal horodaté. On en tire, par tâche, la durée, l'attente de relecture, les allers-retours, l'attention consommée, le temps bloqué et les tokens consommés ; puis des médianes par taille de risque et par type de code, et la calibration de l'attention par taille.
- **La fin se prévoit sans estimer**, à partir du débit réel de tâches fusionnées, par simulation : une date à 50, 85 et 95 % de chances. Sous trois semaines d'historique, la prévision est refusée plutôt que déguisée.

Sprint review et rétrospective

Chaque itération se clôt par une commande dédiée, `/tizi-sprint-review` (l'agent `tizi-sprint-review` dans Copilot), qui prépare tout en une fois à partir des mesures réelles et produit un document en deux parties, comme Scrum les distingue :

- **la sprint review**, avec les parties prenantes, regarde le produit : ce qui a été réalisé et prouvé, puis l'adaptation du backlog — pour chaque tâche non finie, continuer, découper ou reporter ; les tailles à revoir ; la prévision par le débit ;
- **la rétrospective**, entre membres de l'équipe, regarde la façon de travailler : ce que le sprint a coûté comparé au précédent (attente de relecture, allers-retours, attention prévue et consommée, temps bloqué), les causes de blocage récurrentes, chacune avec une action et un responsable, et le suivi des actions décidées au sprint précédent, avec leur effet dans les chiffres.

La capacité suggérée change d'unité : quand les agents codent, elle se compte en tâches fusionnées et en attention humaine, plus en points de taille. Les décisions ne sont enregistrées que lorsque l'équipe les déclare, et la vérification signale celles qui manquent.

Pour qu'elle ne soit pas oubliée, la sprint review est **déclenchée par la date de fin** de l'itération, sans attendre qu'on change un statut : le point d'étape la signale, le **hook de début de session** la rappelle à quiconque ouvre l'agent dans le dépôt, et une **tâche planifiée en CI**, facultative, prépare chaque jour ouvré les sprint reviews dues et ouvre une issue pour toute l'équipe — elle ne décide rien et ne modifie aucun fichier.

Sur une application existante

Tizi ne redocumente jamais toute l'application : il travaille changement par changement, et chaque changement ne cartographie que ce qui le borde. La carte grandit au fil des changements, sans chantier initial.

1. **Adopter, une fois.** Un script observe le dépôt sans rien exécuter — fichiers de projet, schémas Xcode partagés, `package.json`, Prisma, docker-compose, contrats OpenAPI — et propose la configuration des tests et une carte grossière des conteneurs, chacun prouvé par le fichier qui l'établit. Ce qu'il ne peut établir (les échanges entre conteneurs, le simulateur iOS) est listé comme incertain. Il n'écrase jamais une configuration existante.
2. **Choisir le chemin de chaque changement.** La plupart passent par la voie rapide ou le chemin court ; un changement qui touche la sécurité des patients prend la chaîne complète.

3. **Dire ce qu'il ne faut pas casser.** Le PRD d'un changement sur un produit en service relève les comportements existants à préserver, chacun avec la preuve qu'il existe ; ce sont des exigences de non-régression, prouvées par un test à la release.
4. **Concevoir sur l'existant.** La conception reprend la carte d'adoption comme existant à vérifier, établit les échanges à partir du code et distingue l'état actuel de l'état cible ; un contrat consommé par d'autres est une contrainte héritée.
5. **Figer l'existant avant de le modifier.** Sur un élément existant modifié, ou un comportement à préserver, l'implémentation commence par des tests de caractérisation qui décrivent ce que le code fait aujourd'hui et passent tels quels ; le cycle rouge–vert vient ensuite. La porte de PR vérifie cet ordre.

Une chaîne par opportunité

La chaîne BRD → PRD → conception → plan ne se déroule pas pour un produit entier d'un seul tenant : elle tourne **par opportunité**, le plus souvent une fonctionnalité ou une amélioration. On ne cartographie que ce qui sert la conversation. Une chaîne courte qui livre et apprend vaut mieux qu'une longue séquence de documents, car la plupart des idées ne produisent pas l'effet espéré tant qu'on ne les a pas confrontées aux utilisateurs.

1. Recueil du besoin

Objectif : comprendre le problème à résoudre avant de penser à la solution.

Activités

- Entretiens avec les utilisateurs et les parties prenantes (soignants, patients, administrateurs).
- Ateliers d'event storming pour cartographier le domaine métier.
- Observation des usages actuels et identification des irritants.
- Distinction des exigences fonctionnelles et non fonctionnelles (performance, sécurité, accessibilité, langues).

Artefacts produits : notes d'entretien, carte d'event storming, liste brute des exigences, glossaire métier.

Gate : le porteur du besoin valide que la liste d'exigences reflète bien son problème.

Construction du BRD : les quatre phases

Le BRD (Business Requirements Document) se construit en quatre phases : d'abord Assess, qui évalue l'opportunité et décide s'il faut aller plus loin, puis trois phases, avec l'aide d'un agent analyste métier : **Discover** (comprendre), **Define** (structurer les

exigences `BR-xxx`) et **Govern** (revue, approbation, handoff vers le PRD). L'agent accompagne et propose ; l'utilisateur valide et décide. Le contrôle formel du BRD fera l'objet d'une phase ultérieure.

Phase Assess : évaluer l'opportunité

Avant la découverte, une conversation courte décide si l'idée mérite qu'on y investisse. Elle s'appuie sur un canevas d'opportunité en neuf cases : problèmes, utilisateurs et clients, solutions actuelles, valeur pour l'utilisateur, mesures côté utilisateur, adoption, problème de l'organisation, mesures côté organisation, budget en ordre de grandeur. La décision, rendue par le sponsor ou le product owner, est **aller plus loin**, **jeter**, **apprendre d'abord** ou **reporter**. Jeter est un bon résultat. Un script liste le portefeuille de toutes les opportunités et de leurs décisions.

Emprunts au design thinking

Quatre pratiques du design thinking de hve-core complètent la découverte du BRD :

- **Demande figée ou fluide.** La demande initiale est notée mot pour mot et classée. Une demande figée impose une solution précise et masque souvent le problème : on remonte de la solution présumée au problème. Une demande fluide est une aspiration vague qu'il faut d'abord focaliser.
- **Contraintes d'environnement.** Bruit, lumière, mains occupées ou gantées, appareils réellement disponibles, réseau, interruptions, temps disponible, stress : relevées de préférence par l'observation, avec leur implication, puis transmises au PRD comme contexte d'usage.
- **Classe de preuve.** Chaque exigence métier porte une preuve calculée à partir de ses sources : observée (observation, mesure, test), rapportée (entretien, atelier, document), supposée (aucune source) ; une question ouverte la signale non résolue. La vérification de forme alerte quand une exigence repose sur une seule source.
- **Porte de synthèse.** Avant de passer en Define, la synthèse est jugée sur cinq dimensions : fidélité aux sources, couverture des parties prenantes, robustesse, actionnabilité, alignement. Une dimension faible se corrige ou s'assume avec une raison.

Une démarche menée avec le DT Coach de hve-core peut servir de source : un script lit son résumé de passage et propose contraintes, hypothèses et zones d'ombre selon ses marqueurs de confiance (validé, supposé, inconnu, contradictoire), sans rien inscrire dans l'état sans validation.

Enfin, Define ajoute les **exigences de transition** (`TR`) du BABOK : formation, conduite du changement, reprise de données, coexistence, communication, support, chacune avec ses parties prenantes et un critère d'achèvement. Le PRD les rattache à une release, le

plan les planifie.

Phase Discover

Objectif : passer d'une idée floue à une compréhension partagée du problème, en faisant apparaître puis en levant les zones d'ombre, et en collectant des besoins candidats rattachés à leurs sources.

Posture de l'agent

- Il pose 2 à 3 questions à la fois, puis reformule pour valider.
- Il n'invente rien : toute déduction est enregistrée comme hypothèse.
- Il reste neutre vis-à-vis de la solution : face à une solution (« il faut une appli mobile »), il remonte au besoin réel par des « pourquoi ? » successifs.
- Il rend chaque zone d'ombre explicite, avec la personne qui peut la lever.

Les zones d'ombre

Une zone d'ombre est ce qu'on ne sait pas encore et qui pourrait fausser le BRD. C'est le fil conducteur de Discover.

Type	Signal typique	Action pour la lever
Problème flou	Symptômes décrits, cause ou impact non établis	Entretien, données existantes
Partie prenante non entendue	Profil identifié mais pas encore interrogé	Entretien
Profil manquant	Rôle probable jamais cité (support, juridique…)	Question à l'utilisateur
Succès non mesurable	Objectif sans indicateur	Question au sponsor
Contradiction	Deux sources disent l'inverse ; qualifiée en factuelle, vocabulaire, réglementaire ou intérêts	Preuve (factuelle), glossaire (vocabulaire), interprétation juridique (réglementaire), décision de l'arbitre (intérêts)
Hypothèse non vérifiée	Affirmation sans source	Document, donnée, entretien
Solution déguisée	Un « besoin » qui décrit un moyen	Remontée au besoin

Type	Signal typique	Action pour la lever
Périmètre flou	Inclusion ou exclusion non tranchée	Décision du sponsor
Attendu implicite	Ce que l'existant fait « sans que personne n'y pense » n'est couvert par aucune source	Question directe, analyse de l'existant, observation
Arbitre inconnu	Un conflit d'intérêts n'a pas de décideur identifié	Question au sponsor, puis carte des droits de décision

Une zone d'ombre est **bloquante** si les objectifs ou les exigences ne peuvent pas être formulés sans elle. Elle se termine **levée** (une réponse sourcée existe) ou **assumée** (l'utilisateur décide de vivre avec ; elle deviendra une hypothèse ou un risque du BRD).

Les capacités de l'agent

1. **Cadrage** : sept thèmes explorés par petits lots (initiative et déclencheur, problème, signaux de succès, parties prenantes et opposants possibles, périmètre et contraintes, matériaux existants, qui tranche en cas de désaccord). Se termine par une reformulation du problème validée par l'utilisateur et une première carte des droits de décision, confirmée par le sponsor.
2. **Cartographie des parties prenantes** : impact, pouvoir et sens de l'influence (positive, négative, neutre, inconnue), dont se déduit l'implication (impliquer, satisfaire, consulter, informer) ; attitude envers le changement ; distinction entre vrais utilisateurs et représentants ; profils oubliés ; ordre d'interrogation. Les opposants sont entendus individuellement : ce sont des sources, pas des obstacles.
3. **Préparation d'entretien** : un guide par partie prenante, centré sur les zones d'ombre qu'elle peut lever, avec questions ouvertes, besoins exprimés en négatif, exceptions, consigne du silence avant de relancer, et pièges à éviter.
4. **Analyse de source** : à partir de notes, transcriptions ou documents, extraction des irritants, besoins candidats, verbatims, solutions évoquées et règles métier, chacun avec son emplacement exact. Trois filtres repèrent généralisations, omissions et distorsions ; chacune devient une relance ou une zone d'ombre. Une synthèse est renvoyée à la personne interrogée.
5. **Atelier Event Storming** : brief avant l'atelier ; après, reconstitution du processus actuel, conversion des hotspots en zones d'ombre ou en besoins, et enregistrement des règles métier apparues.
6. **Point d'étape** : synthèse du problème, des entretiens, des besoins et des zones d'ombre (bloquantes en premier), avec une prochaine action recommandée.

7. **Council de lentilles** : détection de zones d'ombre par points de vue multiples (voir ci-dessous).
8. **Observation** : voir le travail réel, active ou passive, pour faire apparaître contournements, outils parallèles et attendus implicites. Elle complète l'entretien, sans montrer le travail intellectuel.
9. **Conflit d'intérêts** : l'arbitre est le plus petit niveau qui a autorité sur toutes les parties, jamais l'une d'elles. L'agent prépare une décision avec au moins deux options et leurs impacts, sans recommander ; la décision n'est enregistrée que lorsque l'utilisateur déclare qui l'a rendue.

Le council de lentilles

Plusieurs agents examinent l'initiative, chacun à travers une **lentille** : utilisateur final, opérations et support, sécurité et conformité, sponsor et finance, sceptique.

- **Règles** : une lentille est un point de vue d'analyse, pas une personne ; elle ne produit que des questions, jamais de besoins ; elle ne lève aucune zone d'ombre ; chaque question désigne un interlocuteur réel.
- **Déroulé** : toutes les lentilles reçoivent le même dossier et travaillent en parallèle sans voir les autres. L'agent principal arbitre (fusion des doublons, écarts, typage, 10 questions au plus), puis l'utilisateur retient ce qui est pertinent.
- **Moments** : après le cadrage (mode préparation, pour alimenter les guides d'entretien) et avant la fin de Discover (mode couverture, pour repérer les angles morts restants).
- **Mesure** : chaque passage enregistre le nombre de questions proposées, écartées et retenues, pour évaluer l'utilité réelle du council.

État de session et artefacts

Un fichier `state.yaml` par initiative sert de mémoire de travail et permet de reprendre entre deux entretiens. Il utilise des identifiants stables, jamais réutilisés : `SH` (partie prenante), `SRC` (source), `BC` (besoin candidat), `H` (hypothèse), `ZO` (zone d'ombre).

S'y ajoutent `RG` (règle métier), `DD` (décision) et la carte `decision_rights` : domaine, décideur, recours, qui l'a confirmée.

Artefacts produits : `state.yaml`, cartographie des parties prenantes, guides d'entretien, briefs d'atelier, sources analysées.

Fin de la phase Discover

L'agent **propose** le passage à Define, sans le décider, quand :

- le problème est formulé et confirmé par l'utilisateur ;

- aucune zone d'ombre bloquante n'est ouverte ;
- les parties prenantes à impliquer et à consulter ont été entendues, ou leur absence est assumée ;
- chaque objectif pressenti a au moins un signal de succès ;
- les besoins candidats sont sourcés ;
- un décideur, confirmé par le sponsor, est désigné au moins pour le périmètre, les objectifs et les priorités ;
- aucun conflit d'intérêts n'est ouvert sans arbitre ;
- la question des attendus implicites a été posée pour chaque processus touché ;
- un council en mode couverture a été proposé ;
- la vérification de forme a été lancée, ses avertissements listés sans bloquer.

Phase Define

Objectif : faire converger la matière de Discover vers un BRD structuré : objectifs mesurables, exigences métier reliées à leurs sources, critères d'acceptation, priorités, processus cible, hypothèses et risques. Discover explore ; Define structure et fait décider.

Mode de travail

- L'agent soumet des propositions par petits lots (3 à 5 éléments), que l'utilisateur valide, modifie ou rejette.
- Tout élément est créé au statut `proposé` et ne passe à `validé` qu'avec l'accord explicite de l'utilisateur.
- Rien ne disparaît en silence : un besoin candidat non retenu est marqué `abandonné` ou `hors périmètre`, avec sa raison.
- Aucune exigence n'existe sans au moins un besoin candidat sourcé.

Source de vérité et génération du BRD

Le fichier `state.yaml` fait foi. Le document `brd.md` est **généré** à partir de lui par un script Node déterministe, et écrasé à chaque génération. Toute correction passe par l'agent, qui met à jour l'état puis régénère le document. Ainsi, le document et les données ne peuvent pas diverger, et le contrôle ultérieur du BRD pourra s'appuyer directement sur l'état.

Par défaut, seuls les éléments validés apparaissent dans le BRD. Une option permet de générer un brouillon de travail qui inclut les éléments proposés.

Vérification de forme

Avant de présenter un lot, l'agent lance `lint.mjs` sur les éléments concernés et montre les avertissements avec chacun. Les règles portent sur la rédaction (termes vagues, énoncé multiple, pronom sans antécédent, généralisation, solution dans le BRD, formulation non normative) et sur la structure (champs obligatoires, références inexistantes, arbitre partie au conflit).

Rien n'est bloqué. Valider malgré un avertissement s'enregistre comme dérogation, avec sa raison. Le catalogue `lib/rules.mjs` est commun au BRD et au PRD, et sera repris tel quel, en mode bloquant, par le contrôle formel.

Les étapes de Define

1. **Objectifs métier (`OBJ`)** : construction d'une impact map (Pourquoi → Qui → Comment → Quoi). Chaque objectif a un indicateur, une valeur actuelle, une cible, une échéance et un responsable. Une valeur inconnue n'est jamais inventée : elle devient une zone d'ombre.
2. **Exigences métier (`BR`)** : regroupement des besoins candidats, puis rédaction selon des règles fixes (une exigence par besoin, formulation côté métier, neutre vis-à-vis de la solution). Chaque exigence est reliée à ses objectifs, à ses besoins candidats et aux règles qu'elle fait respecter. Les incohérences de couverture sont signalées sous forme de questions.
3. **Règles métier (`RG`)** : un référentiel séparé des exigences. Chaque règle a un type (définition ou comportement), un niveau d'application pour les règles de comportement (stricte, dérogation autorisée, dérogation justifiée, indicative), un responsable métier, ses sources et sa raison d'être. Une exception est une règle à part. Le choix d'automatiser revient au PRD.
4. **Critères d'acceptation métier** : 1 à 3 résultats observables par exigence, au niveau métier, sans détail technique.
5. **Priorisation MoSCoW** : l'agent propose une priorité justifiée ; le décideur désigné pour les priorités arbitre. L'agent signale quand plus de 60 % des exigences sont en « Must ».
6. **Processus cible** : à partir du processus actuel reconstitué en Discover, chaque événement est marqué nouveau, modifié, inchangé ou supprimé. Les impacts organisationnels sont listés.
7. **Consolidation** : revue des hypothèses, risques (`R`) issus des zones d'ombre assumées, décisions (`DD`), périmètre final, contraintes, dépendances et glossaire.

Le BRD produit

Le document généré comporte dix sections : contexte et problème, objectifs et indicateurs (avec l'impact map), parties prenantes, processus actuel et cible, exigences métier, contraintes, hypothèses et risques, périmètre, glossaire, et une annexe de traçabilité (objectif → exigence → besoin candidat → source). L'impact map et le processus cible sont rendus en diagrammes Mermaid.

Il comprend aussi une section Règles métier, la carte des droits de décision, les décisions avec leurs options et leur décideur, et une annexe des points de vérification avec les dérogations acceptées.

Retour ponctuel en Discover

Si une zone d'ombre bloquante apparaît pendant Define, l'agent propose un retour ciblé sur cette seule zone d'ombre (préparer l'entretien ou la question qui la lèvera), puis reprend l'étape en cours. Chaque retour est tracé.

Council en mode complétude (optionnel)

Après la rédaction des exigences, le council de lentilles peut chercher des objectifs ou des exigences manquants et des critères non observables. Ses résultats restent des questions : une exigence n'est ajoutée qu'après que la question a été levée par une source réelle.

Fin de la phase Define

L'agent **propose** le passage à Govern, sans le décider, quand :

- toutes les sections du BRD sont validées ;
- chaque objectif a au moins une exigence, et chaque exigence au moins un objectif ;
- chaque exigence a au moins un critère d'acceptation et une priorité ;
- chaque règle métier validée a une source, un type et un responsable ;
- chaque besoin candidat confirmé est intégré, abandonné ou hors périmètre ;
- aucune décision n'attend sans zone d'ombre ouverte qui l'explique ;
- aucune zone d'ombre bloquante n'est ouverte ;
- chaque avertissement de vérification est corrigé ou couvert par une dérogation ;
- le BRD a été régénéré après la dernière validation.

Phase Govern

Objectif : faire relire le BRD par les bonnes personnes, intégrer leurs retours de façon traçable, obtenir l'approbation, puis figer une version de référence (baseline) et la transmettre au PRD.

Les règles des phases précédentes restent valables : `state.yaml` fait foi, tout passe par proposition puis validation, rien ne disparaît en silence, et toute nouvelle exigence passe par un besoin candidat sourcé.

Versions

- `0.x` : brouillons et versions candidates soumises à revue.
- `1.0` : première baseline approuvée.
- Après une baseline : version mineure pour un ajustement sans effet sur le périmètre, version majeure pour un changement de périmètre ou d'objectif.

Une version figée est une copie immuable de l'état et du BRD, accompagnée d'un changelog calculé par identifiants (éléments ajoutés, modifiés, retirés).

Les étapes de Govern

1. **Préparer la revue** : les rôles de revue sont déduits de la cartographie et des droits de décision (les décideurs du périmètre et des objectifs, ainsi que les profils à impliquer, approuvent, ceux à consulter relisent, ceux à satisfaire sont informés). Les avertissements de vérification sont traités avant l'envoi, pour que les relecteurs se concentrent sur le fond. La version candidate est figée, puis un dossier de revue est généré : une synthèse d'une page et une fiche par relecteur, avec les exigences issues de ses propres échanges, les questions qui lui sont adressées, les règles métier dont il est responsable et les arbitrages attendus de lui. Diffusion par export (Word, PDF, document partagé) par défaut, ou par pull request.
2. **Collecter les retours** : chaque retour est enregistré comme une source, puis découpé en commentaires (`CM`) classés en correction, modification, nouvelle exigence, contestation ou question.
3. **Traiter les retours** : chaque commentaire reçoit une décision motivée (accepté, rejeté, reporté), ce qui forme la matrice de résolution des commentaires. Une nouvelle exigence passe par un besoin candidat sourcé ; une contestation devient une contradiction qualifiée, tranchée par l'arbitre désigné s'il s'agit d'un conflit d'intérêts, avec un retour ponctuel en Define ou en Discover si elle est bloquante. Si les changements sont significatifs, un nouveau cycle de revue est proposé.

4. **Approuver** : une approbation n'est enregistrée que lorsque l'utilisateur la déclare (qui, quelle version, quelle décision, preuve facultative). Une approbation avec réserves transforme chaque réserve en risque ou en zone d'ombre suivie. Un refus renvoie à l'étape ou à la phase concernée.
5. **Baseline et handoff** : la version approuvée est figée comme baseline (avec un tag Git si possible), puis un paquet de handoff est généré pour le PRD.

Le paquet de handoff vers le PRD

Généré uniquement à partir d'une version figée, en YAML pour l'agent suivant et en Markdown pour un lecteur humain. Il contient le problème, les objectifs, les exigences par priorité avec leurs critères d'acceptation, le processus cible, les contraintes, le périmètre, les hypothèses encore à vérifier, les risques, les zones d'ombre assumées, les réserves d'approbation et le glossaire. Le PRD doit conserver les identifiants `BR` pour préserver la traçabilité.

Il transmet aussi les règles métier validées, la carte des droits de décision et les décisions rendues, que le PRD reprend telles quelles.

Après la baseline : demandes de changement

Toute modification d'un BRD en baseline passe par une demande de changement (`CR`) : enregistrement de la demande, analyse d'impact (éléments touchés, changement de périmètre ou non), décision de l'utilisateur, puis nouveau cycle revue → approbation → baseline sur une nouvelle version.

Scripts de la construction du BRD

Script	Rôle
`render-brd.mjs`	Génère le BRD à partir de l'état
`review-pack.mjs`	Génère la synthèse et les fiches relecteurs
`freeze-version.mjs`	Fige une version immuable, avec changelog
`build-handoff.mjs`	Génère le paquet de handoff vers le PRD
`lint.mjs`	Vérifie la forme, sans bloquer ; utilisé à chaque lot et avant la revue
`list-opportunities.mjs`	Liste le portefeuille des opportunités et de leurs décisions

Script	Rôle
`ingest-dt-handoff.mjs`	Propose, depuis un résumé de design thinking de hve-core, sources, contraintes, hypothèses et zones d'ombre, sans modifier l'état
`choose-path.mjs`	Choisit et enregistre le chemin d'une initiative (commun à tous les skills)

L'agent n'écrit jamais lui-même les fichiers produits par ces scripts. S'ils sont indisponibles, il le signale à l'utilisateur.

Fin de la construction du BRD

La construction du BRD est terminée quand une baseline existe et que le handoff a été généré. L'agent propose alors de passer au PRD, sans le démarrer lui-même.

2. Analyse et spécification

Objectif : transformer les besoins en exigences précises, testables et priorisées.

Activités

- Vision, objectifs produit, personas fondés et parcours utilisateurs.
- Story map et découpage en releases, avec un MVP.
- Exigences fonctionnelles en EARS et critères d'acceptation en Gherkin (Given / When / Then).
- Exigences non fonctionnelles mesurables, contraintes, décisions produit et métriques.

Artefacts produits : PRD généré, story map et releases, FR, critères d'acceptation, NFR, décisions produit, handoff vers la conception et la planification. Les user stories détaillées viennent à l'étape 4.

Gate : le PRD est approuvé et figé en baseline ; chaque FR a une origine, des critères d'acceptation, une release et une priorité. La *Definition of Ready* des stories s'applique à l'étape 4.

Construction du PRD

Le PRD (Product Requirements Document) décrit **ce que le produit fait** : des comportements observables, pour qui et dans quel ordre de livraison. Il n'entre pas dans l'implémentation (architecture, technologies, modèle de données), qui relève de la conception, ni dans les user stories détaillées, qui relèvent de la planification.

Un agent product manager accompagne sa construction ; l'utilisateur valide et décide. Comme pour le BRD, `state.yaml` fait foi et le document est généré par script.

Deux modes d'entrée

Mode	Entrée	Traçabilité
Chaîné	Handoff d'une baseline du BRD	FR / NFR → exigences du BRD → objectifs métier → sources
Autonome	Un brief, des notes ou une idée	FR / NFR → objectifs produit → sources établies au cadrage

En mode chaîné, le BRD fait autorité sur le métier : un écart (besoin absent du BRD, exigence jugée fausse) devient une demande de changement à remonter au BRD. Le mode autonome convient aux évolutions qui ne justifient pas un BRD complet.

Phase Frame : cadrer le produit

1. **Ingestion** : lecture du handoff du BRD (exigences, contraintes, risques, zones d'ombre assumées) ou mini-cadrage métier en mode autonome.
2. **Vision produit** en quelques phrases, sans technologie.
3. **Objectifs produit (`GOAL`)** SMART, dérivés des objectifs du BRD en mode chaîné.
4. **Personas (`PER`)** fondés sur des preuves : un persona sans source devient une zone d'ombre.
5. **Parcours utilisateurs (`PJ`)**, en demandant systématiquement ce qui se passe en cas d'erreur ou d'interruption.
6. **Story map** : activités puis tâches, avec vérification que les exigences Must du BRD sont couvertes.
7. **Releases et MVP (`REL`)** : le MVP doit tester une hypothèse explicite, offrir un parcours complet, rester minimal, définir un signal mesurable, limiter l'impact d'un échec et justifier chaque exclusion.

En mode chaîné, l'ingestion reprend aussi les règles métier et la carte des droits de décision du BRD ; en mode autonome, Frame les établit. Pour chaque étape d'un parcours, l'agent demande ce que l'existant fait déjà sans que personne n'y pense : les utilisateurs taisent les fonctions qu'ils jugent évidentes, et leur absence est ce qui les mécontente le plus.

Avant de spécifier, chaque hypothèse est classée par type (valeur, usage, viabilité, faisabilité) et par risque. Une hypothèse de valeur ou d'usage à **risque fort** doit être **testée ou assumée** : entretien, BD de conception, prototype papier ou cliquable, test

d'usage, service rendu à la main, faux bouton, analytique. Le critère de réussite est écrit avant le test. Le prototype reste un instrument de test, conservé comme source : le PRD continue de décrire des comportements, sans écrans. Sans hypothèse risquée de ce type, l'étape se réduit au classement.

Quand la demande était figée, que le PRD est autonome ou que plusieurs solutions sont défendues, Frame fait d'abord **explorer des concepts** (méthodes 4 et 5 du design thinking de hve-core) : diverger en idées brutes, les regrouper en deux à quatre concepts, les évaluer sur trois axes — désirable, faisable, viable — puis les tester avant que le décideur en retienne un. La story map se construit sur le concept retenu ; les concepts écartés restent avec leur raison. Un concept n'est ni une exigence ni une maquette, et l'axe « faisable » reste une hypothèse vérifiée en conception : la séparation entre le quoi et le comment est préservée. Les supports de test restent volontairement bruts, pour attirer la critique de fond plutôt que des remarques d'apparence.

Sur un produit déjà en service, Frame relève aussi les **comportements existants à préserver** (`PB`) : ce que le produit fait aujourd'hui, que le changement pourrait casser et dont quelqu'un dépend, chacun avec la preuve qu'il existe. Ce sont des exigences de non-régression : une tâche du plan les protège par des tests de caractérisation, et la release les prouve comme toute exigence.

Phase Specify : spécifier

1. **Exigences fonctionnelles (`FR`)** rédigées en EARS (« Quand…, le système doit… », « Si…, alors le système doit… »). Chacune a un acteur, une release, une priorité et une origine (exigence du BRD, source ou décision produit). Une exigence sans origine signale une inflation du périmètre. Chaque lot passe par la vérification de forme avant d'être présenté (termes vagues, détails d'implémentation, énoncé multiple).
2. **Critères d'acceptation (`AC`)** en Gherkin (Étant donné / Quand / Alors), avec un identifiant propre. Les exigences Must ont au moins un scénario nominal et un scénario d'erreur. Ces scénarios alimentent directement les tests de l'étape 6.
3. **Exigences non fonctionnelles (`NFR`)** classées selon l'ISO/IEC 25010, toujours avec une métrique, une cible et un moyen de mesure.
4. **Contraintes (`CON`)** imposées de l'extérieur, distinctes des NFR, et **décisions produit (`DD`)** avec les options écartées et le décideur qui les a rendues, issu de la carte des droits de décision.
5. **Métriques (`MET`)** et événements à instrumenter.
6. **Couverture** : exigences du BRD non couvertes, FR sans critère, objectifs sans métrique, règles automatisées sans exigence, signalés sous forme de questions.

Le PRD décide quelles règles métier automatiser. Une règle automatisée est référencée par les FR qui la font respecter, sans être réécrite, pour qu'un changement de règle se propage par la traçabilité. Son niveau d'application dicte le comportement : stricte → refus avec scénario d'erreur ; dérogation autorisée → seul un acteur habilité passe outre ; dérogation justifiée → le produit demande et conserve la justification ; indicative → simple avertissement.

Phase Govern

Même déroulé que pour le BRD : revue (le product owner approuve ; le porteur métier, la conception et la qualité relisent), traitement des retours, approbation déclarée par l'utilisateur, baseline et handoff. Un commentaire qui remet en cause le BRD devient une demande de changement vers le BRD. Le handoff, organisé release par release, transmet à la conception et à la planification les FR avec leurs critères, les NFR, les contraintes, les décisions produit, l'instrumentation et les points de vigilance. Les identifiants FR, NFR et AC sont conservés en aval.

Zones d'ombre et council

Le PRD a sa propre typologie de zones d'ombre : cas limite, état d'erreur, comportement ambigu, NFR ou objectif non mesurable, persona non fondé, exigence sans origine, exigence du BRD non couverte, conflit avec l'amont, dépendance inconnue, hypothèse non vérifiée.

Quatre types s'y ajoutent : attendu implicite, règle non couverte, conflit d'intérêts et arbitre inconnu. Un conflit d'intérêts se tranche comme dans le BRD, par le décideur désigné, jamais par l'agent.

Un council optionnel examine le PRD à travers six lentilles : UX et accessibilité, testabilité, opérations et support, sécurité et confidentialité, faisabilité, et un sceptique chargé de la discipline de périmètre. Comme pour le BRD, il ne produit que des questions.

Scripts

`render-prd.mjs` génère le PRD avec les scénarios Gherkin et une annexe de couverture. `review-pack.mjs`, `freeze-version.mjs` et `build-handoff.mjs` jouent le même rôle que pour le BRD ; le script de gel de version est commun aux deux documents.

`lint.mjs` et son catalogue `lib/rules.mjs` sont les mêmes que pour le BRD. Le PRD généré comprend une section Règles métier avec leur couverture, les droits de décision et une annexe des points de vérification.

3. Conception / architecture

Objectif : décider comment le système répondra aux exigences, avant d'écrire le code.

Activités

- Modélisation du domaine (agrégats, entités, événements).
- Définition des contrats d'API (OpenAPI) et du modèle de données.
- Choix techniques documentés en ADR (Architecture Decision Records).
- Modélisation des menaces (STRIDE) et conception de la sécurité (authentification, chiffrement, journalisation).

Artefacts produits : modèle de domaine, contrats d'API, schéma de données, ADR, analyse de menaces.

Gate : revue d'architecture validée ; les risques identifiés ont une mesure de maîtrise.

Construction de la conception

La conception est construite avec le skill `sad-builder`, sur le même squelette que le BRD et le PRD : trois phases (**Explore**, **Design**, **Govern**), `state.yaml` comme source de vérité, zones d'ombre, council de lentilles, vérification de forme commune, gel et handoff. Il part du handoff d'un PRD en baseline et ne modifie jamais une FR, une NFR ou une règle : un écart devient une demande de changement vers l'amont.

Trois choix structurent l'étape :

- **Un SAD minimal généré**, et un fichier par décision au format MADR v4. Les ADR vivent plus longtemps que le dossier.
- **Un modèle C4 unique dans l'état**, d'où les vues sont générées en Mermaid (`flowchart` avec conventions C4). Un élément porte ainsi le même nom à tous les niveaux.
- **Un brownfield léger dès la première version** : inventaire prouvé, deux états du modèle, contraintes héritées. Pas de plan de migration détaillé.

Phase Explore

1. **Ingestion du PRD** : FR, NFR, contraintes, règles automatisées, personas et carte des décideurs, complétée des domaines `architecture` et `acceptation_risque_securite`.
2. **Inventaire de l'existant** (brownfield) : limité à ce qui borde le changement. Chaque élément et chaque relation cite une preuve (dépôt, commit, chemin) ; sans preuve, c'est une hypothèse.

3. **Moteurs architecturaux (`AD`)** : les exigences qui changent la structure, exprimées en scénarios de qualité à six parties (source, stimulus, environnement, artefact, réponse, mesure) et cotés en importance et en difficulté.
4. **Domaine (`CTX`)** : Event Storming de niveau conception, contextes délimités, agrégats et carte des contextes. Chaque règle métier automatisée est garantie par un agrégat.
5. **Contexte (C4 niveau 1)** : le système, ses utilisateurs et les systèmes externes.

Phase Design

1. **Décisions (`ADR`)** pour toute question coûteuse à défaire ou qui sert un moteur : au moins deux options, des moteurs, une conséquence négative, la divergence qu'il empêche et la règle vérifiable qui l'assure, un moyen de confirmation, un décideur déclaré. Un ADR accepté n'est jamais réécrit, il est remplacé.
2. **Conteneurs et composants** (C4 niveaux 2 et 3), puis **allocation** de chaque FR à au moins un élément.
3. **Réponses aux NFR (`NR`)** : tactiques, éléments, ADR, et une vérification rattachée à l'étape du SDLC où elle sera faite (tests, intégration, recette, exploitation).
4. **Données (`DAT`)** : propriétaire unique, classification (dont personnelle et santé), conservation.
5. **Interfaces (`API` , `OP`)** : chaque opération sert au moins une FR ; un squelette OpenAPI ou AsyncAPI est généré par interface.
6. **Menaces (`TH`)** : STRIDE sur les flux qui franchissent une frontière de confiance et sur les stockages de données sensibles. Une menace acceptée exige le décideur désigné.

Phase Govern

La revue d'architecture porte sur les décisions et leurs compromis : la synthèse montre les moteurs principaux, ce qui les sert, ce que chaque ADR sacrifie, les menaces acceptées et les hypothèses encore à prouver. Le handoff vers la planification liste les éléments à construire ou modifier avec leurs FR et releases, les contrats, les vérifications par étape, les mesures de sécurité, les spikes à mener et, en brownfield, les éléments remplacés ou retirés.

Brownfield léger

Cycle de vie	Vue actuelle	Vue cible
existant conservé	oui	oui
existant modifié	oui	oui, signalé en orange

Cycle de vie	Vue actuelle	Vue cible
existant remplacé	oui	non, remplaçant désigné
existant retiré	oui	non
nouveau	non	oui, signalé en vert

Toute transition autre que « conservé » est justifiée par un ADR. Un rôle humain sans cycle de vie explicite est considéré comme existant.

Sur une application où Tizi vient d'être adopté, l'inventaire part de la carte grossière proposée par l'adoption (`.tizi/adoption.yaml`) : chaque conteneur y cite le fichier qui l'établit, et la conception le vérifie dans le code avant de le valider. Après chaque version figée, les règles des ADR, les dossiers des composants, les contrats et les données sensibles sont reportés dans les instructions du dépôt (`AGENTS.md`) pour les agents qui ne passent pas par Tizi.

Vérification et zones d'ombre

Le catalogue commun gagne seize règles (S21 à S36) : moteur sans mesure, ADR à option unique ou sans moteur, ADR accepté sans conséquence négative ni confirmation, FR non allouée, NFR sans réponse, existant sans preuve, opération sans FR, donnée sensible sans menace de divulgation examinée, menace sans traitement, nom d'élément dupliqué, règle sans agrégat. Les zones d'ombre propres à l'étape sont : moteur non mesurable, hypothèse technique (à lever par un spike), existant non prouvé, frontière floue, FR non allouée, NFR sans réponse, contrat ambigu, menace non traitée, dépendance inconnue.

Revue d'architecture et référentiels de sécurité

La revue de la phase Govern suit une grille par pilier (fiabilité, sécurité, optimisation des coûts, excellence opérationnelle, efficacité des performances), à la manière du Well-Architected Framework. Le dossier de revue rassemble pour chaque pilier les preuves déjà présentes dans l'état : moteurs, ADR, vérifications, menaces. L'appréciation revient aux relecteurs ; un pilier jugé à risque devient un risque suivi.

Un projet peut retenir des référentiels de contrôles (OWASP ASVS ou MASVS, NIST SP 800-53, CIS Controls). Chaque menace atténuée cite alors les contrôles que ses mesures réalisent, et la vérification de forme signale celles qui n'en citent aucun.

Scripts

Script	Rôle
`render-sad.mjs`	Génère le SAD, les vues C4 actuelles et cibles, un fichier MADR par ADR et les squelettes de contrats
`lint.mjs`	Vérifie la forme, sans bloquer
`review-pack.mjs`	Génère la synthèse de revue d'architecture et les fiches relecteurs
`freeze-version.mjs`	Fige une version avec ses ADR et contrats, et un changelog
`build-handoff.mjs`	Génère le paquet de handoff vers la planification

4. Planification

Objectif : organiser la réalisation en tâches livrables et estimées.

Activités

- Découpage des user stories sous la story map du PRD, spikes en tête, walking skeleton ; voie rapide pour les petites corrections.
- Example mapping de chaque story, qui sert de *Definition of Ready*, et démonstration convenue en atelier.
- Découpage en tâches d'une PR chacune, avec une fiche de contexte mesurée et gelée ; dépendances, contrôle des collisions.
- Taille de chaque tâche lue comme un risque, calculé depuis ses données ; attention humaine prévue ; affectation à un agent ou à un humain.
- Planification en sprint ou en flux, *Definition of Done*, export vers GitHub Issues et Jira.
- Blocages enregistrés au fil de l'eau ; mesures du réel, prévision par le débit ; sprint review et rétrospective.

Artefacts produits : plan généré, fiches de stories avec leur example mapping, fiches de contexte des tâches, backlog exporté, Definition of Done, handoff vers l'implémentation ; registre des blocages, mesures du réel, documents de sprint review.

Gate : l'équipe s'engage sur une itération (sprint) ou un réapprovisionnement (flux) qui ne contient que des stories prêtes.

Construction du plan

Le plan est construit avec le skill `plan-builder`, sur le même squelette que les étapes précédentes : `state.yaml` comme source de vérité, zones d'ombre, council, vérification de forme commune, gel et handoff. Il part des handoffs du PRD et de la conception, copiés en lecture seule dans `refs.yaml` ; l'ingestion signale si la conception et le PRD ne sont pas de la même version.

La story map n'est pas refaite : elle vient du PRD, qui décide quoi livrer et dans quel ordre. L'étape 4 y découpe les stories de la release en cours.

Trois choix structurent l'étape :

- **Un état unique exporté vers GitHub Issues et Jira**, l'outil de suivi n'étant qu'une vue.
- **Sprint ou flux, au choix par projet**, avec le même découpage en stories et en tâches.
- **Des tailles S, M, L, XL lues comme un risque**, puisque les agents réalisent les tâches : une taille calculée depuis les données de la tâche est montrée à côté de celle de l'équipe, chaque tâche porte une attention humaine prévue, et une tâche XL est à découper (voir « Estimer quand les agents codent »).

Les cinq phases

1. **Slice** : stories verticales sous chaque tâche de la story map, chacune rattachée à au moins une FR ; les spikes de la conception passent en tête ; le walking skeleton traverse tous les conteneurs de bout en bout ; chaque story est classée en ouverture, milieu ou fin de partie.
2. **Refine** : un example mapping obligatoire par story, avec au moins le métier, le développement et le test, et la démonstration convenue pour la revue. Les règles viennent du BRD et du PRD, les premiers exemples des critères d'acceptation. Une règle absente de l'amont ou un exemple qui change le comportement remonte en demande de changement.
3. **Schedule** : tâches d'une PR chacune, dépendances dérivées de la conception (fournisseur d'un contrat avant son consommateur, spike avant ce qui en dépend), contrôle des collisions entre tâches sans dépendance qui touchent le même élément ou le même contrat, tailles et attention prévue, affectation, puis itération ou file « prêt ».
4. **Govern** : engagement de l'équipe, baseline, export, handoff vers l'implémentation.
5. **Review** : en fin d'itération, sprint review puis rétrospective, à partir des mesures réelles ; après chaque release, maturité et mesure des outcomes.

Story prête : une Definition of Ready calculée

Une story est prête si et seulement si elle a une FR et une tâche de story map, que l'example mapping a eu lieu, que chaque règle a au moins un exemple, qu'aucune question ni demande de changement n'est ouverte, que ses spikes sont terminés et qu'elle n'est pas XL. Le script le calcule ; personne ne le déclare. Le handoff retient les tâches d'une story non prête, même si elles figurent dans un engagement.

Agent ou humain

Une tâche va à un agent seulement si sa taille ne dépasse pas M, que sa story est prête, qu'elle cite ses éléments et ses contrats, et qu'elle ne demande ni spike ni décision d'architecture. Chaque tâche reçoit une fiche de contexte générée — FR et critères, règles et exemples, éléments touchés, contrats, règles des ADR, mesures de sécurité, vérifications, comportements à préserver, Definition of Done — mesurée en taille et gelée par son empreinte dans le handoff. Quand toutes les tâches sont réalisées par des agents, la limite du débit n'est plus le développement mais l'attention humaine : validation de la compréhension, réponses aux questions, relecture.

Préparer, approuver, exécuter

La planification et l'écriture dans le tracker sont **séparées en deux skills**, comme le Functional Planner et le Backlog Manager de hve-core, mais par des verrous déterministes plutôt que par des consignes.

Étape	Qui	Droits	Produit
Découvrir	`plan-builder`	Lecture seule	Types, champs requis, labels, jalons ou versions, issues existantes
Préparer	`plan-builder`	Aucun accès réseau	Plan d'opérations signé (`operations.yaml` et son empreinte) et sa version à relire
Approuver	L'utilisateur	—	L'empreinte exacte, le niveau d'autonomie et les opérations « à relire » acceptées, consignés dans l'état
Exécuter	`backlog-executor`	Écriture	Issues créées ou mises à jour, table de correspondance, journal

La préparation valide les types et champs contre le tracker, dédoublonne (rattachement d'une issue qui porte déjà l'identifiant, alerte sur un titre très proche), applique six garde-fous au contenu (secrets, chemins locaux et adresses masqués, références non résolues,

éléments non validés exclus, longueurs) et désigne les éléments pas encore créés par des références symboliques que seul l'exécuteur résout.

L'exécuteur refuse un plan dont l'empreinte ne correspond pas, qui n'est pas approuvé, ou qui est périmé : état du plan modifié (hors approbation) ou autre exécution entre-temps. Il applique selon trois niveaux d'autonomie (complet, partiel par défaut, manuel), ignore proprement les opérations qui dépendent d'une opération écartée, s'arrête à la première erreur et journalise tout. Les scripts de planification ne contiennent aucune requête d'écriture, et seuls les jetons d'écriture de l'exécuteur permettent d'écrire.

Scripts

Script	Skill	Rôle
`ingest-handoffs.mjs`	plan-builder	Vérifie l'empreinte des handoffs du PRD et de la conception, puis les copie dans `refs.yaml`
`render-plan.mjs`	plan-builder	Génère le plan, les fiches de stories et les fiches de contexte des tâches, avec risque calculé, attention prévue et taille des fiches
`lint.mjs`	plan-builder	Vérifie la forme et la couverture, sans bloquer
`discover-tracker.mjs`	plan-builder	Découvre le tracker en lecture seule
`plan-operations.mjs`	plan-builder	Prépare le plan d'opérations signé, sans aucune écriture
`freeze-version.mjs`	plan-builder	Fige une version avec ses fiches et `refs.yaml`
`build-handoff.mjs`	plan-builder	Ordonne les tâches engagées et prêtes, avec l'empreinte et la taille de chaque fiche
`block.mjs`	tous	Ouvre et lève les blocages d'une story ou d'une tâche, causes typées
`measure.mjs`	plan-builder	Mesure le réel : durées, attente de relecture, allers-retours, attention, blocages, tokens ; calibration
`forecast.mjs`	plan-builder	Prévoit la fin par le débit réel, par simulation

Script	Skill	Rôle
`close-sprint.mjs`	plan-builder	Prépare la sprint review et la rétrospective, et reporte les fusions dans le plan
`sprint-review.mjs`, `retro-evidence.mjs`	plan-builder	Fiches de revue et preuves sourcées de l'itération
`due-reviews.mjs`, `ci-sprint-review.mjs`	plan-builder	Détectent les sprint reviews dues ; en CI, préparent les documents et ouvrent une issue
`apply-operations.mjs`	backlog-executor	Vérifie les trois verrous, applique selon l'autonomie, journalise

Fin d'itération et après release

Chaque itération se clôt par la sprint review puis la rétrospective, avec la commande `/tizi-sprint-review` (voir « Sprint review et rétrospective »). Elle prépare tout à partir des mesures réelles, fait décider du sort de chaque tâche non finie, traite les causes de blocage récurrentes et suit les actions du sprint précédent ; la capacité suggérée y est comptée en tâches fusionnées et en attention humaine.

Après chaque release :

- **Maturité de release** : avant la date promise, une note de A à F par activité de la story map, pour concentrer l'effort là où c'est le plus faible.
- **Mesure des outcomes** : décidée avant chaque livraison (métriques du PRD, observation d'utilisateurs réels), conclue après ; chaque apprentissage devient une story ou une nouvelle opportunité.

Voie rapide et stratégie de jeu

Les petites corrections évidentes entrent en **voie rapide** : origine observée, taille S ou M, aucune règle métier, aucun critère convenu, aucun contrat ni aucune mesure de sécurité touchés. Elles sont prêtes avec leur origine et un exemple (la reproduction), sans rattachement au PRD ; au-delà de ces bornes, elles suivent le chemin normal.

Chaque release se construit en trois temps classés par l'équipe : **ouverture** (le walking skeleton et le risqué, qui traversent tout), **milieu** (options, règles difficiles, performance), **fin** (affinage). Une story trop grosse s'amincit en bon, mieux, meilleur, et l'on construit les versions « bon » partout avant d'améliorer. Les stories ont un titre court avec qui, quoi et pourquoi ; le modèle « En tant que… » est facultatif.

5. Implémentation

Objectif : produire un code conforme à la conception, lisible et maintenable.

Activités

- Une tâche prête à la fois, sur une branche dédiée, dans l'ordre du handoff du plan.
- Compréhension reformulée par l'agent et validée par un humain, puis gelée.
- Sur du code existant, tests de caractérisation d'abord ; puis scénarios d'acceptation (BDD) et tests de contrat vus en échec ; puis TDD, imposé par un garde-fou.
- Mutation testing incrémental, traçabilité des exemples et critères, analyse de sécurité.
- Revue de code par agents sous plusieurs perspectives, avec vérification adversariale, puis revue humaine obligatoire.

Artefacts produits : code source et tests, journal horodaté de la tâche, rapport de revue par agents, description de PR avec la checklist du relecteur.

Gate : la porte de PR est ouverte (C1 à C9 : compréhension validée et inchangée, aucune question ouverte, suites au vert, TDD tracé, traçabilité complète, mutation au seuil, sécurité propre, revue par agents sur le diff actuel si exigée, existant caractérisé avant modification), la même porte est rejouée en CI, et un humain a approuvé la PR.

Construction de l'implémentation

L'implémentation se fait avec le skill `impl-builder`, tâche par tâche, à partir du handoff du plan vérifié par son empreinte. Une tâche n'est prise que si ses dépendances sont fusionnées.

1. **Comprendre** : l'agent reformule la tâche, pose ses questions, liste ses hypothèses et son plan de tests ; un humain valide, et la compréhension est gelée par son empreinte. Une question qui touche une règle, un critère, un contrat ou un ADR bloque la tâche et repart en amont ; si le chemin choisi ne prévoit pas l'étape qui doit trancher, on l'élargit.
2. **Caractériser** (sur du code existant) : figer le comportement actuel par des tests qui passent tels quels, avant toute modification.
3. **Rouge** : les scénarios d'acceptation des exemples et critères de la tâche (étiquetés `@EX-xxx`, `@AC-xxx`, `@PB-xx`) et les tests de contrat sont écrits d'abord et vus en échec, puis le premier test unitaire.
4. **Vert, refactoriser** : le minimum de code pour passer, puis l'amélioration sur du vert, test après test.
5. **Vérifier** : traçabilité, mutation testing incrémental, sécurité, revue de code par agents.
6. **PR** : description et checklist de revue générées ; la revue d'une PR d'agent est toujours humaine, et chaque décision (approbation, corrections, fusion) est enregistrée avec son heure.

Un blocage extérieur (environnement, accès, données, réponse attendue) est enregistré au moment où il survient, et levé quand il cesse.

La discipline est imposée par un **hook, pour Claude Code comme pour GitHub Copilot**, qui bloque toute écriture tant que la compréhension n'est pas validée ou si elle a changé depuis, tout code de production sans test en échec enregistré, toute écriture hors des dossiers des éléments C4 de la tâche, et toute modification de contrat que la tâche ne prévoit pas. La même porte est rejouée en CI sur chaque pull request, pour ce qui échappe au hook (un éditeur sans hooks, une écriture par commande shell). Un test trivial ou une refactorisation détournée peuvent tromper le hook ; le mutation testing et la revue les rattrapent.

Avant la revue humaine, une **revue de code par agents** examine le changement sous quatre perspectives — fonctionnelle (FR, critères, exemples, règles), architecture et contrats (ADR, opérations, périmètre), sécurité (menaces, zones à risque, données sensibles), tests (assertions, mutants survivants) — chacune en contexte neuf, sans voir les autres. Elle est rattachée au commit et à l'empreinte exacte du diff : si le code change, elle est périmée. Chaque constat porte un emplacement, l'attendu, le constaté, une preuve et la plus petite correction, **sans aucun score de confiance** ; ceux qui en contiennent sont nettoyés et signalés. Tout constat bloquant ou majeur passe par un **vérificateur adversarial** qui cherche à le réfuter : confirmé, réfuté avec la preuve contraire, ou rétrogradé. Le verdict — vérification incomplète, corrections requises, ou prête pour la relecture humaine — peut être exigé par la porte de PR. Méthode reprise de la revue de code et du vérificateur de constats de hve-core.

Deux disciplines encadrent la tâche avant le code. La **fiche de contexte** est mesurée (budget de taille, au-delà duquel l'agent perd en précision) et **gelée** par son empreinte dans le handoff du plan : on ne la corrige pas à la main. La **compréhension validée** est gelée de même : si elle change après validation, le garde-fou bloque les écritures et la porte de PR reste fermée jusqu'à une nouvelle validation. La revue de code compte en outre une lentille des **manques de vérification** — un test manquant n'est signalé que si l'on nomme la régression qu'il attraperait — et une lentille des **affirmations**, qui lit ce que la tâche déclare avoir fait comme un témoignage, après avoir tracé le code. Idées reprises de BMAD-METHOD.

Script	Rôle
`start-task.mjs`	Prend une tâche prête : handoff et fiche vérifiés par leur empreinte, dépendances fusionnées, périmètre fixé pour le hook
`approve-understanding.mjs`	Enregistre la validation humaine de la compréhension et la gèle

Script	Rôle
`run-tests.mjs`	Lance une suite, enregistre rouge ou vert (ou caractérisation) pour le hook et le journal
`tdd-guard.mjs`	Le hook : autorise ou refuse chaque écriture
`tdd-phase.mjs`	Ouvre une refactorisation sur du vert
`trace-check.mjs`	Vérifie que chaque exemple, critère et comportement à préserver de la tâche a un test
`review-context.mjs`, `review-merge.mjs`	Préparent la revue par agents sur le diff exact, puis fusionnent et vérifient les constats
`pr-gate.mjs`	La porte de PR (C1 à C9) et la description de PR ; avec `--run`, la rejoue en CI
`record-pr.mjs`	Enregistre l'approbation, les corrections demandées et la fusion, avec leur heure
`adopt.mjs`	Sur une application existante : propose la configuration des tests et la carte de l'existant

6. Tests

Objectif : prouver que le logiciel fait ce qui est spécifié, et rien de dangereux.

Activités

- Suites unitaires et d'acceptation complètes en non-régression ; intégration réelle entre composants déployés (par exemple app iOS et API Next.js).
- Tests de bout en bout, peu nombreux, sur les parcours critiques.
- Sessions de tests exploratoires minutées, avec une charte « explorer X avec Y pour découvrir Z », menées par des humains et tirées des risques connus.
- Tests non fonctionnels : charge et endurance, DAST, audit d'accessibilité, multi-navigateurs et multi-langues.
- Mutation testing global, pour juger la qualité des tests plutôt que la seule couverture de code.

Artefacts produits : matrice de couverture des exigences (réalisé et prouvé, écarts), relevé de provenance des preuves, rapports de sessions exploratoires, archive des preuves de la version candidate avec son manifeste d'empreintes.

Gate : la porte de release est ouverte (R1 à R10 : suites au vert, exigences prouvées ou dérogations nommées, NFR conformes, parcours critiques couverts, exploratoire fait sans bloquant, mutation et sécurité au niveau, rien de réalisé sans être prouvé, preuves issues de la CI sur le commit candidat, aucun défaut bloquant ou majeur ouvert ni signalement à trier), et un humain a approuvé la release.

Construction de la vérification

La vérification d'une release se fait avec le skill `test-builder`, en trois phases : **Prepare** (exigences à prouver, parcours critiques, chartes exploratoires, vérifications non fonctionnelles, environnements), **Execute** (suites en CI sur une version candidate identifiée, sessions exploratoires, mesures) et **Prove**.

Les tests de la boucle courte (unitaires en TDD, acceptation en BDD, contrats) sont écrits à l'étape 5 avec le code ; l'étape 6 les exécute à l'échelle de la release et ajoute ce qu'une tâche seule ne peut pas vérifier.

La **matrice de couverture des exigences** est calculée, jamais déclarée : chaque exemple, critère, vérification de NFR, mesure de sécurité et parcours critique de la release est relié aux tests qui portent son identifiant dans les rapports collectés. La porte de release est ouverte quand toutes les suites passent, que chaque exigence est prouvée ou couverte par une dérogation nommée, que les NFR atteignent leurs cibles, que les parcours critiques passent de bout en bout, que les sessions exploratoires sont faites sans constat bloquant, et que le score de mutation et les analyses de sécurité sont au niveau. Les preuves d'une version candidate sont archivées une seule fois, en lecture seule, avec un manifeste d'empreintes SHA-256.

Réalisé et prouvé. La matrice croise deux faits, chacun calculé depuis sa seule source : la **réalisation** (de chaque FR à ses stories, à leurs tâches, et à l'état de chacune — fusionnée ou terminée) et la **preuve** (les tests qui portent l'identifiant et passent). Elle les présente par activité de la story map, la vue que comprennent les parties prenantes, et liste trois écarts. **Réalisé mais non prouvé** ferme la porte : une tâche fusionnée montre qu'un code existe, pas que le comportement convenu fonctionne ; il faut écrire le test, retirer de la release, ou déroger par décision nommée. **Non réalisé** ne ferme pas la porte mais appelle une décision explicite — reporter, ou retenir la livraison — plutôt qu'une découverte le jour de la livraison. **Prouvé sans être prévu** signale un test qui renvoie à une exigence qu'aucune story de la release ne porte. La porte est un verdict, pas encore un verrou : elle ne bloquera la livraison que lorsque la chaîne de déploiement vérifiera qu'elle est ouverte.

Des preuves probantes, issues de la CI. Un rapport de tests vert ne prouve rien s'il vient d'un poste de travail ou d'une autre version. La CI produit donc les preuves sur le commit de la version candidate lui-même — jamais sur le commit de fusion temporaire d'une pull request — avec un manifeste de provenance : commit, passage, déclenchement,

empreinte de chaque rapport. La vérification retrouve ce passage en lecture seule, contrôle qu'il a réussi en entier, que le manifeste porte sur le bon commit et que chaque rapport utilisé est bien celui-là. Sinon, la porte reste fermée (R9) ; une équipe sans CI peut lever cette condition, mais seulement par une raison écrite, visible dans la matrice. Un audit vérifie enfin la CI elle-même : porte de PR rejouée et exigée avant fusion, preuves produites sur le commit, permissions minimales, pas de déclencheur dangereux, actions épinglées ; la protection de branche qu'il ne peut pas lire est déclarée non vérifiable plutôt que supposée.

Script	Rôle
`ci-run-suites.mjs`	En CI : lance les suites, regroupe les rapports, écrit le manifeste de provenance
`ci-fetch.mjs`	Retrouve en lecture seule le passage de CI sur le commit candidat, le vérifie, télécharge les rapports et conserve le manifeste
`record-nfr.mjs`	Enregistre une mesure de NFR prise hors CI, avec le commit mesuré, l'auteur et l'empreinte de son rapport
`release-matrix.mjs`	Matrice réalisé et prouvé, vue par épopée et story, défauts, écarts, porte de release (R1 à R10)
`freeze-evidence.mjs`	Archive une fois les preuves de la version candidate — rapports, mesures, provenance, manifeste de la CI, commit — en lecture seule, avec leurs empreintes ; `--check` vérifie qu'elle est intacte et commitée
`ci-audit.mjs`	Vérifie la configuration de la CI du dépôt
`defect.mjs`	Registre des défauts : signaler, trier, rattacher la correction
`find-expectations.mjs`	Retrouve les attentes écrites proches d'une observation, avec leur texte exact, leur épopée et leur preuve
`import-defects.mjs`	Reprend en lecture seule les bugs ouverts de GitHub ou Jira, comme signalements à trier

Où sont les preuves. Pendant un passage de CI, dans `tizi-preuves/` avec leur manifeste ; ensuite, dans l'artefact du passage, qui **expire au bout de 90 jours** par défaut ; pendant la vérification, dans une copie téléchargée, non versionnée ; après l'approbation, dans

l'archive de la version candidate, `qa/<initiative>/evidence/<version>/`. C'est la seule copie durable, à condition d'être commitée : un logiciel réglementé doit pouvoir montrer ses preuves de vérification des années plus tard. Les mesures que la CI ne peut pas produire — une charge sur un environnement dédié — portent le commit mesuré et l'empreinte de leur rapport, et la porte vérifie qu'elles concernent bien la version candidate.

Signaler et trier un problème. Ce qu'un testeur remonte n'est pas toujours un défaut. La commande `/tizi-bug` (l'agent `tizi-bug` dans Copilot) recueille l'observation — version, environnement, données, étapes — puis retrouve dans toute la spécification les attentes écrites les plus proches, avec leur texte exact, leur épopée et l'état de leur preuve. Le testeur tranche alors par trois questions : une attente écrite couvre-t-elle la situation (sinon, c'est une **question**, à trancher par le PRD) ? le comportement la contredit-il (un **défaut**, rattaché à cette attente, avec une sévérité) ? l'attente paraît-elle fausse (une **demande de changement**, adressée au décideur désigné) ? Les doublons et les cas non reproductibles se ferment comme tels. Un bug déjà saisi dans GitHub ou Jira se reprend en lecture seule plutôt que d'être ressaisi.

Un défaut se corrige par une story en voie rapide, rattachée à l'épopée de l'exigence qu'il contredit, et commence par un test de régression étiqueté `@DEF-xxx` ; il est **résolu quand ce test passe** sur la version candidate, jamais déclaré résolu à la main. Tant qu'il est ouvert, l'exigence qu'il contredit n'est plus « prouvée » mais **contredite** ; si elle l'était, la matrice signale que le test était insuffisant. La porte de release reste fermée (R10) tant qu'un défaut bloquant ou majeur est ouvert, ou qu'un signalement n'est pas trié. La matrice présente enfin la preuve **par épopée et par story**, et l'export peut reporter l'état de preuve de chaque story dans le gestionnaire de tickets.

7. Intégration continue

Objectif : vérifier automatiquement chaque changement avant qu'il rejoigne la branche principale.

Activités

- Pipeline CI sur chaque pull request : build, tests, analyse de sécurité, et la **porte de PR de Tizi rejouée** pour la tâche citée dans le titre ou la branche.
- Sur la branche principale et les tags : **production des preuves de release** sur le commit lui-même, avec leur manifeste de provenance.
- Protection de la branche principale : vérifications exigées avant fusion, relecture humaine.
- Workflows aux permissions minimales et aux actions épinglées ; **audit de la CI** pour le vérifier.

Tizi génère ces workflows (`install.mjs --ci` : porte de PR, preuves, sprint reviews dues) ; l'installation des dépendances du projet et l'épinglage des actions restent à faire par l'équipe.

Artefacts produits : pipelines versionnés, artefact de preuves avec son manifeste, rapport d'audit de la CI.

Gate : merge autorisé uniquement si toutes les quality gates sont au vert.

8. Recette et release

Objectif : confirmer avec le métier que la version livrée répond au besoin.

Activités

- Recette fonctionnelle (UAT) sur un environnement de staging proche de la production.
- Campagne QA manuelle ou pilotée sur les parcours critiques.
- Versioning sémantique (MAJOR.MINOR.PATCH) et rédaction des release notes.
- Vérification que chaque exigence de la version est couverte par un test passé.

Artefacts produits : PV de recette, release notes, tag de version, dossier de release.

Gate : go / no-go signé par le responsable produit (et qualité si dispositif médical).

9. Déploiement

Objectif : mettre la version en production sans interruption et pouvoir revenir en arrière.

Activités

- Déploiement continu (CD) : staging, puis production.
- Exécution des migrations de base de données, réversibles si possible.
- Stratégie de mise en production : blue/green, canary ou feature flags.
- Plan de rollback testé et smoke tests après déploiement.

Artefacts produits : pipeline CD, journal de déploiement, procédure de rollback.

Gate : smoke tests de production au vert ; sinon rollback automatique.

10. Exploitation et monitoring

Objectif : garder le service fiable et apprendre de son usage réel.

Activités

- Centralisation des logs, métriques et traces.

- Alertes sur les indicateurs clés (erreurs, latence, disponibilité).
- Gestion des incidents et postmortems sans recherche de coupable.
- Collecte des retours utilisateurs et des anomalies, qui redeviennent des besoins (retour à l'étape 1).

Artefacts produits : tableaux de bord, runbooks, rapports d'incident, nouvelles entrées de backlog.

Gate : les objectifs de service (SLO) sont tenus ; tout incident majeur donne lieu à un postmortem.

Exigences transverses pour un dispositif médical

Pour un logiciel comme Diabeo, ces exigences s'appliquent à toutes les étapes, pas à une seule.

- **IEC 62304** : cycle de vie du logiciel de dispositif médical. Elle fixe une classe de sécurité (A, B ou C) qui détermine les livrables obligatoires à chaque étape.
- **ISO 14971** : gestion des risques, de l'étape 2 (identification) à l'étape 10 (surveillance après commercialisation).
- **Matrice de traçabilité** : chaque exigence est reliée à sa conception, à son code et à ses tests. C'est la pièce centrale lors d'un audit.
- **Hébergement HDS** : les données de santé doivent être hébergées chez un hébergeur certifié HDS. À vérifier pour l'offre OVH utilisée.
- **Gestion des changements** : toute modification après release repasse par l'analyse d'impact et de risques.

Index des abréviations et identifiants

Abréviations

Abréviation	Signification	Où
ADR	Architecture Decision Record : décision d'architecture documentée	Étape 3
BABOK	Business Analysis Body of Knowledge (IIBA)	Sources
BDD	Behavior-Driven Development : scénarios Given / When / Then automatisés	Étapes 5 et 6

Abréviation	Signification	Où
BRD	Business Requirements Document : exigences métier	Étape 1
C4	Modèle d'architecture en quatre niveaux (contexte, conteneurs, composants, code)	Étape 3
CI	Intégration continue	Étapes 5 à 7
DAST	Test dynamique de sécurité de l'application en fonctionnement	Étape 6
DDD	Domain-Driven Design	Étape 3
DoD	Definition of Done : ce qui fait qu'une tâche est finie	Étape 4
DoR	Definition of Ready : ce qui fait qu'une story est prête	Étape 4
EARS	Easy Approach to Requirements Syntax : forme de rédaction des FR	Étape 2
HDS	Hébergement de données de santé	Étapes 2 et 3
ISO 29148	Norme ISO/IEC/IEEE 29148, qualité des exigences	Contrôle formel
KPI	Indicateur clé de performance	Étapes 1, 2 et 10
MADR	Markdown Architectural Decision Records (format d'ADR)	Étape 3
MoSCoW	Must, Should, Could, Won't : priorisation	Étapes 1 et 2
MVP	Minimum Viable Product : plus petite release qui atteint les outcomes visés	Étape 2
PO	Product Owner	Étapes 2 et 4
PR	Pull request	Étape 5
PRD	Product Requirements Document : exigences produit	Étape 2
SAD	Software Architecture Document : dossier d'architecture	Étape 3
SAST	Analyse statique de sécurité du code	Étapes 5 et 6

Abréviation	Signification	Où
SHA-256	Empreinte qui garantit qu'un fichier n'a pas été modifié	Handoffs, exports, preuves
STRIDE	Usurpation, altération, répudiation, divulgation, déni de service, élévation de privilèges	Étape 3
TDD	Test-Driven Development : test écrit avant le code	Étape 5
UX	Expérience utilisateur	Étapes 2 et 6
WIP	Work in progress : limite d'encours en flux	Étape 4

Identifiants par étape

Préfixe	Désigne	Étape
`SH`	Partie prenante	1
`SRC`	Source (entretien, document, atelier, observation, test, code)	1 à 3
`BC`	Besoin candidat	1
`H`	Hypothèse	1 à 3
`ZO`	Zone d'ombre	1 à 5
`OBJ`	Objectif métier	1
`BR`	Exigence métier (business requirement)	1
`RG`	Règle métier	1 et 2
`DD`	Décision (arbitrage au BRD, décision produit au PRD)	1 et 2
`R`	Risque	1 à 4
`RV`, `CM`, `CR`	Cycle de revue, commentaire, demande de changement	1 à 4
`GOAL`	Objectif produit	2
`PER`	Persona	2
`PJ`	Parcours utilisateur	2 et 6
`ACT`, `T`	Activité et tâche de la story map	2 et 4

Préfixe	Désigne	Étape
`REL`	Release	2 et 4
`FR`	Exigence fonctionnelle (functional requirement)	2
`AC`	Critère d'acceptation (Gherkin)	2, 5 et 6
`NFR`	Exigence non fonctionnelle	2 et 3
`CON`	Contrainte	2 et 3
`MET`	Métrique produit	2 et 4
`AD`	Moteur architectural (scénario de qualité)	3
`CTX`	Contexte délimité (bounded context)	3
`PS`, `SYS`, `CT`, `CP`	Personne, système, conteneur, composant du modèle C4	3
`LNK`	Relation entre éléments C4	3
`ADR`	Décision d'architecture	3
`NR`	Réponse de conception à une NFR, avec sa vérification	3 et 6
`DAT`	Donnée métier classifiée	3
`API`, `OP`	Interface et opération de contrat	3 et 5
`TB`	Frontière de confiance	3
`TH`	Menace (STRIDE)	3 et 6
`DT`	Dette technique	3
`US`	User story	4
`EX`	Exemple d'example mapping	4 à 6
`TK`	Tâche (une PR)	4 et 5
`IT`, `RP`	Itération (sprint), réapprovisionnement (flux)	4
`EXE`	Opération d'un plan d'export vers GitHub ou Jira	4
`XS`	Session de tests exploratoires	6
`TR`	Exigence de transition (formation, conduite du changement, reprise de données…)	1, 2 et 4
`CPT`	Concept de solution envisagé avant la story map	2

Préfixe	Désigne	Étape
`PB`	Comportement existant à préserver (non-régression)	2, 4, 5 et 6
`BL`	Blocage d'une story ou d'une tâche, avec sa cause	4 et 5
`DEF`	Défaut signalé, trié et rattaché à l'exigence qu'il contredit	4 à 6

Codes des vérifications et des portes

Codes	Signification
`L01` – `L08`	Règles de rédaction de la vérification de forme (termes vagues, énoncé multiple, pronom sans antécédent…)
`S01` – `S78`	Règles de structure de la vérification de forme, communes à tous les skills
`G1` – `G6`	Garde-fous de contenu de l'export (secrets, chemins locaux, adresses, références non résolues, éléments non validés, longueurs)
`C1` – `C9`	Conditions de la porte de PR (étape 5)
`R1` – `R10`	Conditions de la porte de release (étape 6)

Sources

Les étapes 1 et 2 ont été confrontées en octobre 2026 à deux ouvrages de référence, lus en entier. Le tableau indique d'où vient chaque pratique ajoutée ou confirmée.

Ouvrages de référence

- Yves Constantinidis, *Expression des besoins pour le SI — Guide d'élaboration du cahier des charges*, Eyrolles, collection Solutions d'entreprise. Édition de 2010 lue ; réédition parue le 11 janvier 2018.
- International Institute of Business Analysis (IIBA), *A Guide to the Business Analysis Body of Knowledge (BABOK Guide), version 3*, Toronto, 2015 ; annonce de parution du 15 avril 2015.

Correspondance avec le SDLC

Pratique du SDLC	Constantinidis	BABOK v3
Zones d’ombre comme fil conducteur du recueil	ch. 4, zones d’ombre d’un cahier des charges	technique Item Tracking
Cartographie impact, pouvoir, influence ; opposants entendus	ch. 6 et 7, analyse des parties prenantes	3.2 Plan Stakeholder Engagement ; technique Stakeholder List, Map, or Personas
Droits de décision, arbitre jamais partie au conflit	ch. 8, cohérence au plus tôt ; ch. 18, comité de contrôle des changements	3.3 Plan Business Analysis Governance ; ch. 9, négociation
Filtres de discours, besoins en négatif, silence	ch. 8, recueil et techniques d’interview	4.2 Conduct Elicitation
Attendus implicites	ch. 10, diagramme de Kano	6.1 Analyze Current State
Observation	ch. 8, techniques de recueil	technique Observation
Retour à la source après entretien	ch. 8, feedback rapide	4.3 Confirm Elicitation Results
Référentiel de règles métier, niveaux d’application	ch. 10, règles métier	technique Business Rules Analysis
Vérification de forme pendant la rédaction	ch. 13, check-list de bonne formulation	7.2 Verify Requirements
Validation distincte, faite par d’autres que l’auteur	ch. 15, validation	7.3 Validate Requirements
Toilettage avant revue ; chaque remarque traitée ou rejetée avec justification	ch. 15, revue formelle	3.3 Plan Business Analysis Governance ; technique Reviews
Approbation, y compris avec réserves	ch. 15, signature finale	5.5 Approve Requirements
Identifiant jamais modifié ni réutilisé	ch. 18, attributs d’une exigence	3.4 Plan Business Analysis Information Management
Baseline et demandes de changement avec analyse d’impact	ch. 18, gestion des exigences	5.4 Assess Requirements Changes

Pratique du SDLC	Constantinidis	BABOK v3
Traçabilité exigence ↔ source ↔ objectif	ch. 18	5.1 Trace Requirements

Le modèle de Kano, la *quality gateway* et le modèle Volere des Robertson, ainsi que la notion de validation authentique de Michel Volle, sont cités d'après Constantinidis.

Autres méthodes employées

Ces références sont données de mémoire, sans vérification en ligne pour cette révision : EARS (Mavin et al., 2009) pour la rédaction des FR ; Gherkin (Cucumber) pour les critères d'acceptation ; ISO/IEC 25010 pour les catégories de NFR ; Impact Mapping (Gojko Adzic, 2012) ; User Story Mapping (Jeff Patton, 2014) ; Event Storming (Alberto Brandolini) ; MoSCoW (DSDM).

Pour l'étape 3 : le modèle C4 de Simon Brown (CC BY 4.0) ; le format MADR v4 (CC0 et MIT) et la pratique des ADR de Michael Nygard (2011) ; les scénarios de qualité de Bass, Clements et Kazman, *Software Architecture in Practice* ; le Domain-Driven Design d'Eric Evans (2003) et l'Event Storming d'Alberto Brandolini ; la classification STRIDE de Microsoft (Kohnfelder et Garg, 1999). Ces références sont reformulées, jamais reproduites ; ATAM et arc42 sont seulement cités.

L'étape 3 a aussi été comparée au dépôt microsoft/hve-core, lu au commit du 20 octobre 2026 : ADR Creator (MADR v4), System Architecture Reviewer, Security Planner (STRIDE par famille de composants) et skill c4-architecture (Mermaid, éléments fondés sur des preuves). hve-core n'a pas de dossier d'architecture unique ; le SDLC en génère un à partir du même modèle que les ADR et les vues.

Pour l'étape 4, également données de mémoire : l'example mapping de Matt Wynne (Cucumber, 2015), le User Story Mapping de Jeff Patton (2014) déjà employé au PRD, le walking skeleton d'Alistair Cockburn, et la gestion de backlog du BABOK v3 (technique Backlog Management : juste assez de travail sur chaque élément, au dernier moment responsable).

Emprunts à hve-core intégrés après comparaison (même commit du 20 octobre 2026) : l'empreinte SHA-256 des handoffs et les seuils de couverture avec dérogations (BRD et PRD Builder), la grille ISO/IEC/IEEE 29148 notée de 0 à 3 avec quatre caractéristiques bloquantes (BRD Quality Reviewer), la séparation entre planification en lecture seule et exécution par plateforme, la validation des types et champs, le dédoublonnage, les garde-fous de contenu et les niveaux d'autonomie (Functional Planner, Backlog Manager, skill backlog-management), la revue par piliers (System Architecture Reviewer) et le rattachement à des référentiels de contrôles (Security Planner). Les normes et

référentiels cités (ISO/IEC/IEEE 29148, Microsoft Azure Well-Architected Framework, OWASP ASVS et MASVS, NIST SP 800-53 Rév. 5, CIS Controls) le sont sans reproduction de leur texte.

Jeff Patton, *User Story Mapping: Discover the Whole Story, Build the Right Product*, O'Reilly, 2014, lu en entier en octobre 2026. Il a apporté : la chaîne par opportunité et la phase d'évaluation avec son canevas (ch. 11 et 13), les tests de valeur et d'usage avant de spécifier (ch. 3 et 15), la revue d'équipe, la revue avec les parties prenantes, la maturité de release et la mesure des outcomes (ch. 9, 11 et 18), la voie rapide (ch. 17), la stratégie ouverture–milieu–fin et le découpage bon–mieux–meilleur (ch. 4 et 16), le titre court avec qui, quoi et pourquoi et la démonstration convenue en atelier (ch. 6, 7 et 16). La proposition de faire entrer l'ingénierie et les esquisses d'interface dans le PRD a été écartée : la séparation entre le quoi et le comment est conservée.

Le design thinking de hve-core (agent DT Coach et skills `dt-*`, même commit du 20 octobre 2026) a apporté : le classement de la demande figée ou fluide (méthode 1), les contraintes d'environnement relevées sur le terrain (méthodes 1 et 2), la validation de synthèse en cinq dimensions (méthode 3), l'exploration de concepts sur les axes désirable, faisable et viable (méthodes 4 et 5), le principe de finition minimale des supports de test (contraintes de qualité), et les classes de preuve et marqueurs de confiance (modèle de preuves et contrat de passage vers RPI), repris dans la classe de preuve calculée et le pont d'import. Les exigences de transition viennent du BABOK v3 ; le design thinking (méthode 9) en a confirmé le besoin.

Les contrôles de hve-core (même commit) ont apporté : la suite d'évaluation d'équivalence avec la référence, transposée en évaluation par ablation ; la règle « erreur d'intégrité plutôt que faux succès » des évaluations ; la revue de code multi-perspectives rattachée à un commit ; le vérificateur adversarial des constats ; l'interdiction des scores de confiance dans les rapports ; le principe « le sous-agent propose, le parent vérifie » ; les règles de cohérence des ADR sur les espaces réservés et les chiffres non sourcés ; la mention de relecture humaine dans les documents produits ; et le traitement de tout contenu importé comme donnée.

BMAD-METHOD (version 7 en préversion, commit du 18 octobre 2026) a apporté : la mesure du déclenchement des skills par marqueur, avec séparation entraînement/test fixe ; le budget de taille et le gel de ce que l'agent reçoit (fiche de contexte, compréhension validée) ; les lentilles de revue « manques de vérification » et « affirmations lues comme un témoignage » ; les décisions d'architecture formulées comme des règles qui empêchent une divergence ; le contrôle des collisions entre tâches ; et la rétrospective fondée sur les preuves du dépôt. N'ont pas été repris : le nombre minimal de constats imposé à un relecteur, la boucle d'implémentation sans surveillance, et les personas.